

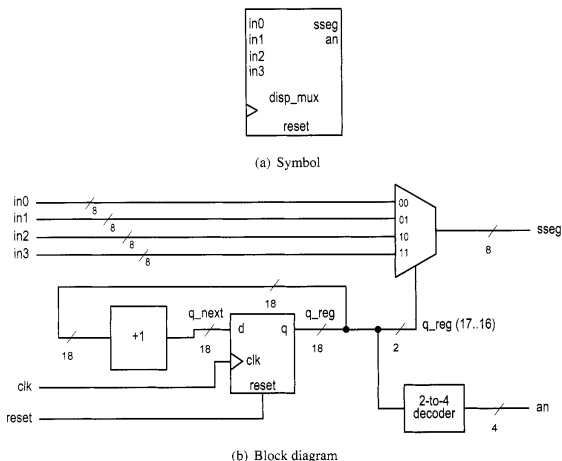


Figure 4.7 Symbol and block diagram of a time-multiplexing circuit.

Time multiplexing with LED patterns The symbol and block diagram of the time-multiplexing circuit are shown in Figure 4.7. It takes four seven-segment LED patterns, in_3 , in_2 , in_1 , and in_0 , and passes them to the output, $sseg$, in accordance with the enable signal.

The refresh rate of the enable signal has to be fast enough to fool our eyes but should be slow enough so that the LEDs can be turned on and off completely. The rate around the range 1000 Hz should work properly. In our design, we use an 18-bit binary counter for this purpose. The two MSBs are decoded to generate the enable signal and are used as the selection signal for multiplexing. The refreshing rate of an individual bit, such as $an(0)$, becomes $\frac{50M}{2^{18}}$ Hz, which is about 800 Hz. The code is shown in Listing 4.13.

Listing 4.13 LED time-multiplexing circuit with LED patterns

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity disp_mux is
5   port (
        clk, reset: in std_logic;
        in3, in2, in1, in0: in std_logic_vector(7 downto 0);
        an: out std_logic_vector(3 downto 0);
        sseg: out std_logic_vector(7 downto 0)
10  );
end disp_mux ;

```

```

architecture arch of disp_mux is
    -- refreshing rate around 800 Hz (50MHz/2^16)
15    constant N: integer:=18;
    signal q_reg, q_next: unsigned(N-1 downto 0);
    signal sel: std_logic_vector(1 downto 0);
begin
    -- register
20    process(clk,reset)
    begin
        if reset='1' then
            q_reg <= (others=>'0');
        elsif (clk'event and clk='1') then
25            q_reg <= q_next;
        end if;
    end process;

    -- next-state logic for the counter
30    q_next <= q_reg + 1;

    -- 2 MSBs of counter to control 4-to-1 multiplexing
    -- and to generate active-low enable signal
    sel <= std_logic_vector(q_reg(N-1 downto N-2));
35    process(sel,in0,in1,in2,in3)
    begin
        case sel is
            when "00" =>
                an <= "1110";
                sseg <= in0;
40            when "01" =>
                an <= "1101";
                sseg <= in1;
            when "10" =>
                an <= "1011";
                sseg <= in2;
45            when others =>
                an <= "0111";
                sseg <= in3;
50        end case;
    end process;
end arch;

```

We use the testing circuit in Figure 4.8 to verify operation of the LED time-multiplexing circuit. It uses four 8-bit registers to store the LED patterns. The registers use the same 8-bit switch as input but are controlled by individual enable signal. When we press a button, the corresponding register is enabled and the switch pattern is loaded to that register. The code is shown in Listing 4.14.

Listing 4.14 Testing circuit for time multiplexing with LED patterns

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity disp_mux_test is
5    port(

```